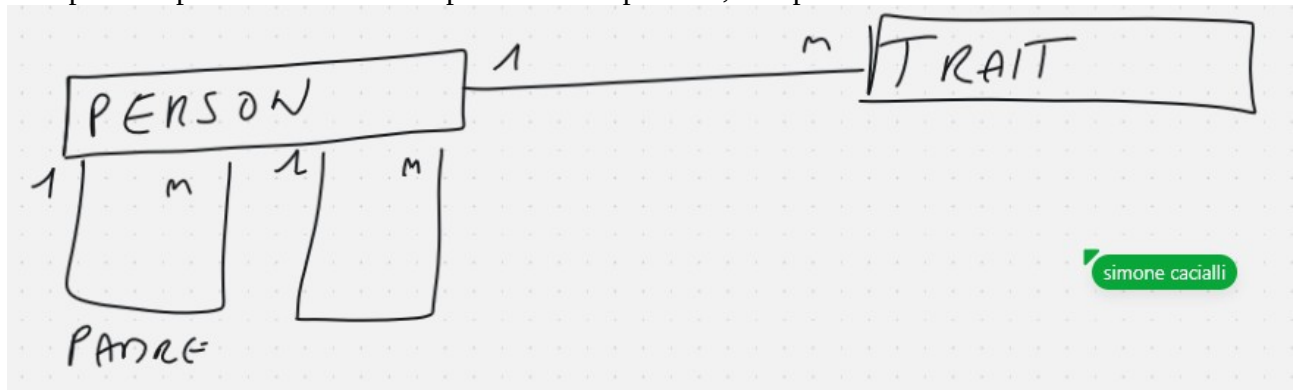


Una persona potrà essere madre e padre di altre persone, una persona ha dei tratti ereditabili



Questo è il data model con una semplificazione su person-trait che in teoria è n – n

Voglio far diventare i dati disponibili in spring

Ho scelto di non avere la lista dei figli all'interno di PersonDTO. Posso avere più di un DTO per la stessa entità.

```

8 import org.springframework.data.jpa.repository.Query;
9
10 import com.generation.people.model.Person;
11
12 public interface PersonRepository extends JpaRepository<Person, Integer>{
13
14     @Query("SELECT p FROM Person p WHERE p.father.id = :fatherId AND p.mother.id = :motherId")
15     List<Person> findByMotherIdAndFatherId(int fatherId, int motherId);
16
17     // 1
18     // abbiamo stabilito che la minia può essere editaria
  
```

Spring scrive metodi per fatti suoi in base al nome del metodo, ma non le può fare tutte, dobbiamo anche essere in grado di scrivere le query a mano, usando due linguaggi: SQL o JPQL

JPQL (Java Persistence Query Language) è un linguaggio di query orientato agli oggetti, parte delle specifiche JPA, che ti permette di interrogare le entità Java invece di scrivere direttamente SQL sulle tabelle del database.

	SQL	JPQL
Opera su	Tabelle e colonne del DB	Entità Java e loro attributi (oggetti)
Dipende dal DB	Sì (MySQL, PostgreSQL...)	No — è portabile
Trasformazione	Nessuna	JPA/Hibernate lo converte in SQL
Sintassi	SELECT * FROM person	SELECT p FROM Person p

La sintassi non è identica, posso usare le relazioni

Hibernate traduce i concetti JAVA in DB

SELF JOIN

Perché JPQL è più elegante qui

In SQL devi fare un **self-join manuale** (alias child e father) perché il DB non conosce la relazione tra le righe. In JPQL invece, grazie al mapping @ManyToOne sull'entità Person, puoi

navigare direttamente la relazione con `p.father.firstName` — Hibernate genera il JOIN automaticamente.

Una **self join** è un JOIN in cui una tabella viene unita con sé stessa. Si usa quando una tabella ha una **relazione ricorsiva** — cioè quando alcune righe fanno riferimento ad altre righe della stessa tabella.

Quando serve

Il caso classico è esattamente il tuo: la tabella `Person` ha una colonna `father_id` che punta a un altro `id` nella **stessa tabella**. Il DB non "sa" che quella colonna rappresenta un padre — per lui sono solo due righe della stessa tabella, quindi devi fare un join esplicito con alias.

//nome di un tratto con tutti gli individui che lo possiedono

```
List<Person> findSonsOfFather(String fatherName);  
  
@Query("SELECT p FROM Person p where :traitName in (select t.name from Trait t where t.person.id = p.id)")  
List<Person> findByTrait(String traitName);  
  
// 1
```

TRAIT DI V

Si poteva usare anche il join

```
@Query("SELECT p FROM Person p JOIN p.traits t WHERE t.name = :traitName")  
List<Person> findByTrait(String traitName);
```

Spring Data JPA		← livello più alto (astrazione)
JPA		← specifica standard Java
Hibernate / EclipseLink		← implementazione concreta
JDBC / SQL		← livello più basso

esercizio: JPQL liscio o con spring data jpa o scriver dei metodi java
fare anche `getPersonByID` e DTO che mandi padre/madre e figli